

Model Context Protocol

& GitHub Copilot



Connecting AI to Your Enterprise Tools, Data Sources, and Services

What is MCP?

The open standard for connecting AI to external systems

Think of MCP as **"USB-C for AI"** — One protocol that connects any AI model to any data source or tool.

● Open Standard

Created by Anthropic (2024); governed by the Linux Foundation's Agentic AI Foundation since Dec 2025

● Universal Connectivity

One protocol connects AI to databases, APIs, file systems & enterprise tools

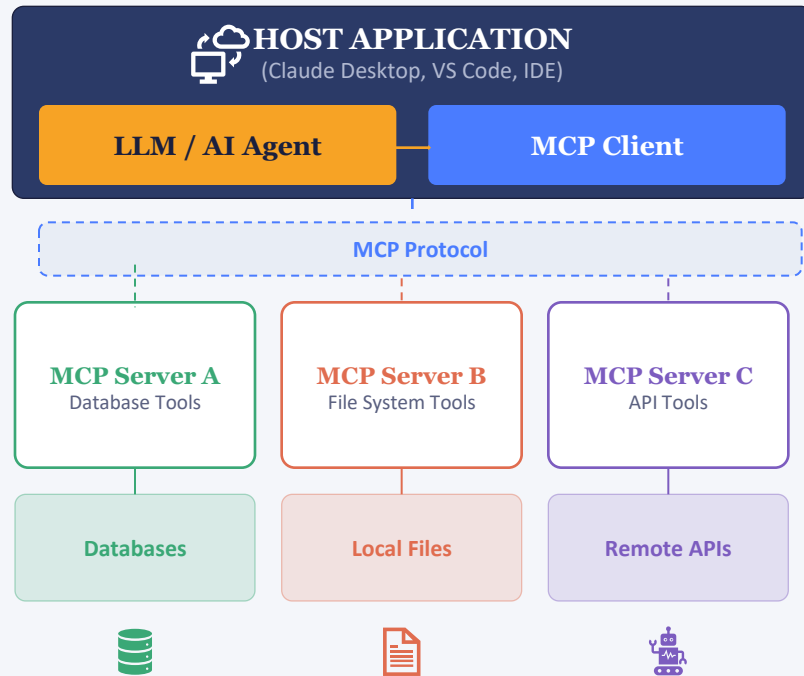
● Security Built-In

Fine-grained permissions & enterprise policies for every connection

● AI Model Agnostic

Works with GPT, Claude, Gemini
— any LLM supported by Copilot

MCP Server Architecture



Why MCP Matters

Four pillars that make MCP essential for enterprise AI adoption



Access Enterprise Systems

Connect AI to databases, APIs, internal tools, and legacy systems without custom integrations



Standardized Integration

One protocol replaces dozens of bespoke connectors — build once, connect everywhere



Security & Control

Enterprise admins control which MCP servers are allowed, with fine-grained policies and trust verification



Context Enrichment

AI gets real-time data from your systems — eliminating hallucination by grounding responses in facts

Without MCP, every AI integration requires custom code. With MCP, it's plug and play.

Without MCP vs With MCP

The same question — drastically different outcomes

Without MCP

1

User → Copilot → GPT:
"What should I wear in Paris?"

2

GPT has no access to real-time weather data

3

Result: AI might hallucinate
"It's warm in Paris" (actually 10°C!)

With MCP

1

User → Copilot:
"What should I wear in Paris?"

2

GPT sees weather tool available:
"Tell me Paris temperature"

3

MCP Server → GetTemperature(Paris)
Returns: 10°C

4

GPT + real data = accurate advice:
"Wear a warm jacket and layers"

MCP Architecture Overview

Four key components working together



MCP Communication Flow

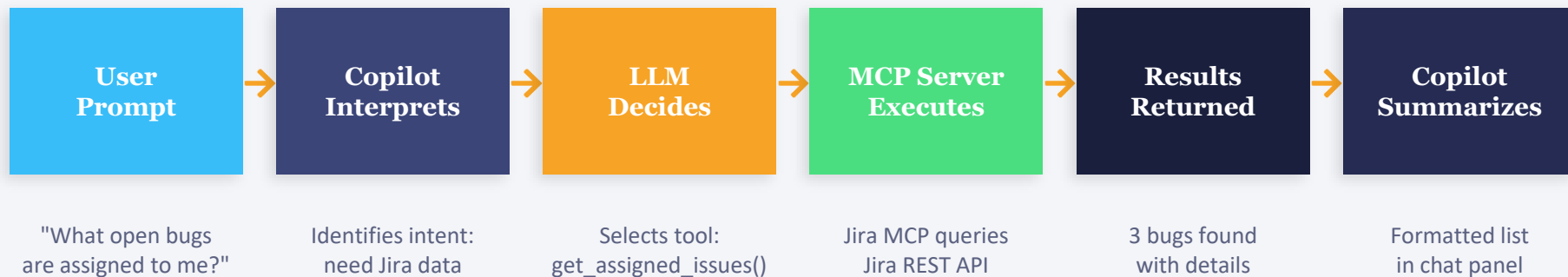
From user prompt to grounded response — step by step

Example: Jira Integration via MCP

Prompt: "What open bugs are assigned to me?"

Copilot → LLM decides: call

`get_assigned_issues(user=@me, status=open)` → Jira MCP Server queries Jira REST API → Returns 3 bugs with priority, title, and sprint info → Copilot displays formatted results in chat



GitHub MCP Server

Deep integration with repos, PRs, issues, commits, and code search

Capabilities

- Repository management & code browsing
- Issues & PR automation (create, update, manage)
- Code search across your organization
- CI/CD & Actions workflow monitoring
- Security: Dependabot alerts, code scanning
- Invoke Copilot Coding Agent via MCP

Key Tools

<code>search_code</code>	<code>list_pull_requests</code>
<code>create_pull_request</code>	<code>get_issue_details</code>
<code>get_file_contents</code>	<code>create_issue</code>

Example Prompt:

"Analyze this repo and find issues in product creation logic."

Available to all GitHub users **regardless of plan type**. Remote (hosted by GitHub, recommended) or Local (on your machine). Supported in VS Code, Visual Studio, JetBrains, Eclipse, Xcode.

GitHub MCP Registry

Your central hub for discovering and installing MCP servers

What It Is

- Curated catalog of MCP servers — now available via docs.github.com and the MCP Registry
- Partner & community-contributed servers
- One-click install from VS Code Extensions view
- Browse: Ctrl+Shift+X → type @mcp in search
- Works with any MCP-compatible host

Partner Ecosystem

Figma: Design-to-code via Dev Mode MCP

Postman: API platform inside your IDE

Terraform: Infrastructure management

Dynatrace: Observability & performance insights

Azure DevOps: Work items, pipelines, docs

How to Browse & Install

- 1 Open Extensions (Ctrl+Shift+X)
- 2 Type @mcp in search bar
- 3 Browse available MCP servers
- 4 Click Install → choose User or Workspace

MCP Configuration: mcp.json

Configure MCP servers for your workspace or project

`.vscode/mcp.json`

```
{
  "servers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot
        .com/mcp/",
      "headers": {
        "Authorization":
          "Bearer ${input:github_pat}"
      }
    }
  },
  "inputs": [{
    "type": "promptString",
    "id": "github_pat",
    "description": "GitHub PAT",
    "password": true
  }]
}
```

Server Types

Remote (http)

Hosted externally — recommended

Local (stdio)

Runs on your machine via command

IDE Support (GA + Preview)

- ✓ VS Code 1.102+ (GA)
- ✓ Visual Studio 17.14+
- ✓ JetBrains IDEs (preview)
- ✓ Eclipse & Xcode (preview)

Share MCP config with your team: commit `.vscode/mcp.json` to your repo

Enterprise MCP Governance

Admin controls for security, compliance, and standardization

Setting MCP Policies

- 1 Enterprise Settings → AI Controls tab → MCP
- 2 Enable "MCP servers in Copilot" (disabled by default)
- 3 Set MCP Registry URL (your custom registry or Azure API Center)
- 4 Choose access policy: **Allow All** / **Register Only** / **Block All**

Key Notes

- Policy applies to Business & Enterprise plans
- Free/Pro/Pro+ not governed by org policy
- Azure API Center provides managed registry
- Coding Agent MCP: configure in repo settings
-

Coding Agent + MCP

Configure MCP servers for Coding Agent in: Repository Settings → Copilot → Coding Agent → MCP Configuration. Supports Azure DevOps MCP, Sentry, and custom servers. Agent uses tools autonomously without per-action approval.

Real-World Enterprise Scenarios

Multi-server MCP workflows across different team contexts

DevOps: Deploy to Staging

Servers: Shell + GitHub + REST API

"Deploy latest main to staging"

Shell: run build → GitHub MCP: create release tag → REST API: trigger deployment webhook

QA: Run Regression Suite

Servers: Shell + Database + Filesystem

"Run full regression and report"

Shell: execute test suite → DB: query test history → Filesystem: write report.html

PM: Sprint Status

Servers: GitHub + Jira + Docs

"What's blocking Sprint 4?"

GitHub: check open PRs → Jira: get blocked items → Docs: find related decisions

Security: Audit Check

Servers: GitHub + Database + Shell

"Audit auth endpoints"

GitHub: scan code for auth patterns → DB: check permission tables → Shell: run security lint

MCP Security & Best Practices

Trust, permissions, and operational guidelines



Trust Verification

Only add MCP servers from trusted sources. VS Code shows confirmation dialog before starting any new server. Review publisher and config.



Secrets Management

Never hardcode API keys. Use input variables or environment files. Coding Agent secrets must be prefixed with `COPILOT_MCP_`.



Least-Privilege Tools

Enable only the tools you need. Use the Tools picker in Chat to toggle servers/tools on and off. Disable unused servers.



Enterprise Policies

Use 'Registry Only' mode for strict environments. Configure allowlists at enterprise or org level. Azure API Center for managed governance.



Tool Limit Awareness

Max 128 tools per chat request (model constraint). If you exceed this, deselect some tools or enable dynamic toolset discovery.



Audit & Review

Review Coding Agent actions via PR commits. MCP server output visible in IDE logs. Monitor tool usage patterns for security.

Local MCP servers can run arbitrary code on your machine — always review before trusting

Key Takeaways

- MCP is the "USB-C for AI" — one open standard connecting any AI model to any data source
- Architecture: MCP Host (IDE) → Client (Copilot) → Server (tools) → External Services
- Without MCP: AI hallucination | With MCP: grounded in real data
- 7 server types: GitHub, Filesystem, Shell, Database, Vector DB, Docs, REST API
- GitHub MCP Registry: curated catalog with one-click install from VS Code
- Multi-server orchestration: LLM decides which tools to chain automatically
- Enterprise governance: Registry Only policies, Azure API Center, Coding Agent MCP